

Answer Key

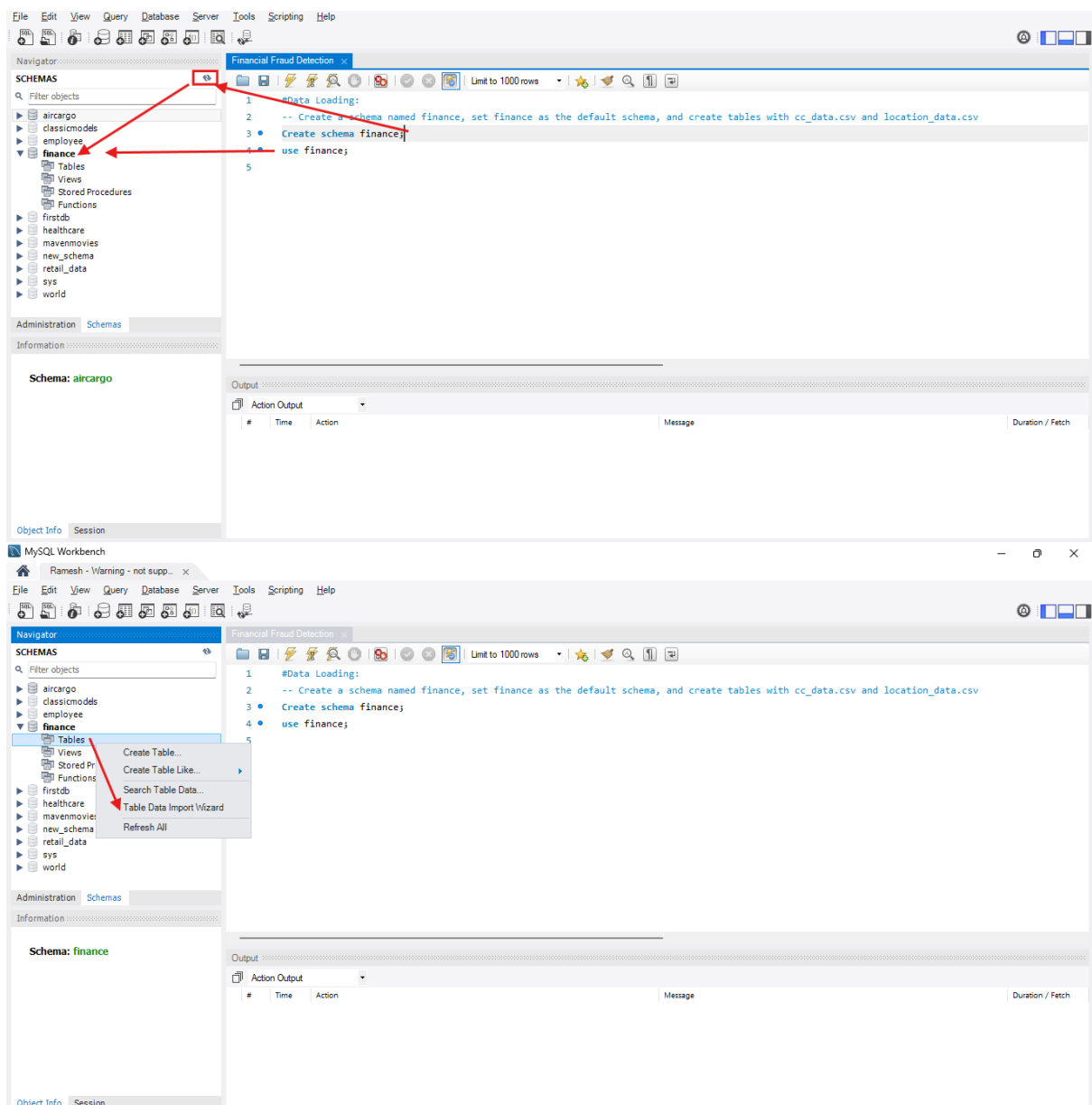
Capstone 1: Financial Fraud Detection

SQL Tasks:

3. Data Loading:

- 3.1 Create a schema named finance, set finance as the default schema, and create tables with cc_data.csv and location_data.csv

Output:



Note: All fields were temporarily imported as TEXT to prevent datatype mismatch errors.

The 'Table Data Import' window is shown with the 'Configure Import Settings' tab active. The detected file format is 'csv' and the encoding is 'utf-8'. A list of columns is displayed on the left, each with a 'Field Type' dropdown menu. Red arrows point to the 'Field Type' dropdown for the 'trans_date_trans_time' column, which is currently set to 'text'. Below the column list, a preview of the data is shown as a table with 16 columns: index, trans_date..., cc_num, merchant, category, amt, first, last, gender, street, city, state, zip, lat, long, city_pop, job, and dob. The data preview shows several rows of transaction records.

index	trans_date...	cc_num	merchant	category	amt	first	last	gender	street	city	state	zip	lat	long	city_pop	job	dob
151888	23-03-2019...	35860844...	fraud_Ferry...	misc_net	124	Crystal	Fuller	F	000 Jennife...	Issaquah	WA	98027	47.4974	-122.0107	50489	Osteopath	29-1
1186025	10-05-2020...	467172701...	fraud_Bogs	grocery_pos	112.57	Kenneth	Edwards	M	3653 Ryan...	Andrews	IN	46702	40.8618	-85.6967	2304	Retail banker	25-1
10818	07-01-2019...	377993105...	fraud_Mohr...	shopping_net	6.69	Nathan	Martinez	M	586 Thoma...	Oconto Falls	WI	54154	44.8755	-88.1555	5548	Mining engi...	11-1
975275	30-01-2020...	471082643...	fraud_Lan...	personal c...	100.69	Juan	Henry	M	9785 Lori L...	Turner	MT	59542	48.8328	-108.3961	192	Further edu...	04-1

The screenshot shows a database IDE with a 'Financial Fraud Detection_SQL' window. The left sidebar shows a 'SCHEMAS' tree with 'finance' selected. The main window displays a SQL script with the following content:

```
1 #Data Loading:
2 -- Create a schema named finance, set finance as the default schema, and create tables with cc_data.csv and location_data.csv
3 Create schema finance;
4 use finance;
5
6 -- 4. Data Exploration with SQL:
7 -- • Calculate the total number of transactions in the cc_data table
8
9 -- • Identify the top 10 most frequent merchants in the cc_data table
10 -- • Find the average transaction amount for each category of transactions in the cc_data table
11 -- • Determine the number of fraudulent transactions and the percentage of total transactions that they represent
12 -- • Join the cc_data and location_data tables to identify the latitude and longitude of each transaction
13 -- • Identify the city with the highest population in the location_data table
14 -- • Find the earliest and latest transaction dates in the cc_data table
15
16
17
```

4. Data Exploration with SQL:

4.1 Calculate the total number of transactions in the **cc_data** table

Financial Fraud Detection_SQL*

```

1  #Data Loading:
2  -- Create a schema named finance, set finance as the default schema, and create tables with cc_data.csv and location_data.csv
3  • Create schema finance;
4  • use finance;
5
6  -- 4. Data Exploration with SQL:
7  → • Calculate the total number of transactions in the cc_data table
8  • SELECT COUNT(*) AS total_transactions FROM cc_data;
9
10 -- • Identify the top 10 most frequent merchants in the cc_data table
11
12 -- • Find the average transaction amount for each category of transactions in the cc_data table
13 -- • Determine the number of fraudulent transactions and the percentage of total transactions that they represent

```

Result Grid

total_transactions
41525

Schema: finance

Object Info Session Result 12 x Read Only

4.2 Identify the top 10 most frequent merchants in the **cc_data** table

Financial Fraud Detection_SQL*

```

9
10 → • Identify the top 10 most frequent merchants in the cc_data table
11 • SELECT merchant, COUNT(*) AS transaction_count FROM cc_data
12 GROUP BY merchant
13 ORDER BY transaction_count DESC
14 LIMIT 10;
15
16 -- • Find the average transaction amount for each category of transactions in the cc_data table
17
18 -- • Determine the number of fraudulent transactions and the percentage of total transactions that they represent
19 -- • Join the cc_data and location_data tables to identify the latitude and longitude of each transaction

```

Result Grid

merchant	transaction_count
fraud_Kilback LLC	148
fraud_Cormier LLC	138
Fraud_Schumm PLC	116
fraud_Erdman-Kertzmann	115
fraud_Sporer Inc	112
fraud_Dickinson Ltd	111
fraud_Kuhn LLC	111
fraud_Luettgen PLC	111
fraud_Rooby, Conn and Tremblay	110
fraud_Stroman, Hudson and Erdman	110

Schema: finance

Object Info Session Result 13 x Read Only

4.3 Find the average transaction amount for each category of transactions in the **cc_data** table

Financial Fraud Detection_SQL*

```

15
16 → • Find the average transaction amount for each category of transactions in the cc_data table
17 • SELECT category, AVG(amt) AS avg_transaction_amount FROM cc_data
18 GROUP BY category;
19
20 -- • Determine the number of fraudulent transactions and the percentage of total transactions that they represent
21
22 -- • Join the cc_data and location_data tables to identify the latitude and longitude of each transaction
23 -- • Identify the city with the highest population in the location_data table
24 -- • Find the earliest and latest transaction dates in the cc_data table
25

```

Result Grid

category	avg_transaction_amount
misc_net	85.76525912408752
grocery_pos	117.8839506891869
shopping_net	83.60911445485945
personal_care	46.65249480396906
gas_transport	63.382279283738884
entertainment	64.28019470404988
home	58.803433501357055
food_dining	51.474271566327914
kids_pets	57.63007282563463
travel	150.49928489702506
misc_pos	60.94562170087968
shopping_pos	82.15103084247566
health_fitness	54.010488590604005
grocery_net	54.09394216133943

Schema: finance

Object Info Session Result 14 x Read Only

4.4 Determine the number of fraudulent transactions and the percentage of total transactions that they represent

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
19
20 -- Determine the number of fraudulent transactions and the percentage of total transactions that they represent
21 • SELECT SUM(is_fraud) AS fraud_transactions, ROUND((SUM(is_fraud) / COUNT(*) * 100, 2) AS fraud_percentage
22 FROM cc_data;
23
24 -- Join the cc_data and location_data tables to identify the latitude and longitude of each transaction
25
26 -- Identify the city with the highest population in the location_data table
27
28 -- Find the earliest and latest transaction dates in the cc_data table
29
```

The results grid shows the following data:

fraud_transactions	fraud_percentage
370	0.59

The left sidebar shows the database schema with tables: cc_data, location_data, Views, Stored Procedures, Functions, firsttab, healthcare, mavenmovies, new_schema, retail_data, sys, and world. The bottom left shows the table structure for cc_data:

Table: cc_data

Columns:

- index text
- trans_date_trans_time text
- cc_num text
- merchant text
- category text
- amt text
- first text
- last text
- number text

4.5 Join the **cc_data** and **location_data** tables to identify the latitude and longitude of each transaction

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
23
24 -- Join the cc_data and location_data tables to identify the latitude and longitude of each transaction
25 • SELECT c.trans_date_trans_time, c.cc_num, c.merchant, l.lat, l.long FROM cc_data c
26 JOIN location_data l
27 ON c.cc_num = l.cc_num;
28
29 -- Identify the city with the highest population in the location_data table
30
31 -- Find the earliest and latest transaction dates in the cc_data table
32
33
```

The results grid shows the following data:

trans_date_trans_time	cc_num	merchant	lat	long
13-12-2019 12:36	60416207185	fraud_Schaefer Ltd	43.0048	-108.8964
11-03-2019 04:16	60422928733	fraud_Friesen Inc	33.6028	-81.9748
11-02-2019 09:40	60416207185	fraud_Cartwright-Harris	43.0048	-108.8964
21-04-2020 03:36	60422928733	fraud_Cummerata-Jones	33.6028	-81.9748
19-03-2019 17:47	60427851591	fraud_Rau-Grant	35.3492	-99.188
14-09-2019 18:51	60422928733	fraud_Reichel, Bradtkie and Blanda	33.6028	-81.9748
30-09-2019 14:32	60495593109	fraud_Hirthe-Beier	32.7699	-96.743
11-01-2019 03:09	60423098130	fraud_Prohaska-Murray	36.9412	-98.2458
03-03-2019 08:38	60490596305	fraud_Sporer Inc	34.8838	-90.7666
14-04-2020 14:24	60416207185	fraud_Jast-McDermott	43.0048	-108.8964
31-08-2019 20:59	60487002085	fraud_Deckow-Diare	32.3739	-90.1293
10-05-2020 06:01	60422928733	fraud_Luettgen PLC	33.6028	-81.9748
07-03-2019 00:54	60490596305	fraud_Harber Inc	34.8838	-90.7666
24-07-2019 06:06	60422928733	fraud_Ernard Inc	33.6028	-81.9748

The left sidebar shows the database schema with tables: cc_data, location_data, Views, Stored Procedures, Functions, firsttab, healthcare, mavenmovies, new_schema, retail_data, sys, and world. The bottom left shows the table structure for location_data:

Table: location_data

Columns:

- cc_num text
- lat text
- long text

4.6 Identify the city with the highest population in the **cc_data** table

The screenshot shows a database IDE with a Navigator pane on the left displaying a schema tree. The main editor contains two SQL queries. The first query identifies the city with the highest population in the `location_data` table. The second query finds the earliest and latest transaction dates in the `cc_data` table. The Results pane at the bottom shows the output of the first query, displaying a table with two columns: `city` and `city_pop`. The result row shows `Pomona` with a population of `9993`.

```

28
29 -- * Identify the city with the highest population in the location_data table
30 * SELECT city, MAX(city_pop) AS highest_population FROM cc_data
31 GROUP BY city
32 ORDER BY highest_population DESC
33 LIMIT 1;
34
35 -- * Find the earliest and latest transaction dates in the cc_data table
36
37
38
39
40
41
42

```

Table: `location_data`

Columns:

cc_num	text
lat	text
long	text

Result Grid

city	city_pop
Pomona	9993

4.7 Find the earliest and the latest transaction dates in the `cc_data` table

The screenshot shows the same database IDE with a different SQL query. This query finds the earliest and latest transaction dates in the `cc_data` table. The Results pane at the bottom shows the output, displaying a table with two columns: `earliest_date` and `latest_date`. The result row shows the dates `01-01-2019 00:06` and `31-12-2019 23:41`.

```

34
35 -- * Find the earliest and latest transaction dates in the cc_data table
36 * SELECT
37 MIN(trans_date_trans_time) AS earliest_date,
38 MAX(trans_date_trans_time) AS latest_date
39 FROM cc_data;
40
41
42
43
44
45
46
47

```

Table: `location_data`

Columns:

cc_num	text
lat	text
long	text

Result Grid

earliest_date	latest_date
01-01-2019 00:06	31-12-2019 23:41

5. Data Aggregation with SQL:

5.1 What is the total amount spent across all transactions in the `cc_data` table?

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```

40
41 -- 5. Using Data Aggregation with SQL:
42 -- • What is the total amount spent across all transactions in the cc_data table?
43 • SELECT SUM(amt) AS total_amount_spent FROM cc_data;
44
45 -- • How many transactions occurred in each category in the cc_data table?
46
47 -- • What is the average transaction amount for each gender in the cc_data table?
48
49 -- • Which day of the week has the highest average transaction amount in the cc_data table?
50
51
52
53
54

```

The results pane shows the following table:

total_amount_spent
7317401.119999985

The left sidebar shows the database schema with the following tables: aircargo, classicmodels, employee, finance, cc_data, location_data, Views, Stored Procedures, Functions, firstdb, healthcare, mavenmovies, new_schema, retail_data, sys, and world.

5.2 How many transactions occurred in each category in the **cc_data** table?

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```

44
45 -- • How many transactions occurred in each category in the cc_data table?
46 • SELECT category, COUNT(*) AS transaction_count FROM cc_data
47   GROUP BY category;
48
49 -- • What is the average transaction amount for each gender in the cc_data table?
50
51 -- • Which day of the week has the highest average transaction amount in the cc_data table?
52
53
54
55

```

The results pane shows the following table:

category	transaction_count
misc_net	5300
grocery_pos	10072
shopping_net	8092
personal_care	7563
gas_transport	10903
entertainment	7595
home	10136
food_dining	7451
kids_pets	9392
travel	3389
misc_pos	6664
shopping_pos	9587
health_fitness	7140

The left sidebar shows the database schema with the following tables: aircargo, classicmodels, employee, finance, cc_data, location_data, Views, Stored Procedures, Functions, firstdb, healthcare, mavenmovies, new_schema, retail_data, sys, and world.

5.3 What is the average transaction amount for each gender in the **cc_data** table?

Financial Fraud Detection_SQL*

Limit to 1000 rows

48

49 → • What is the average transaction amount for each gender in the cc_data table?

50 • `SELECT gender, AVG(amt) AS avg_transaction_amount FROM cc_data`

51 `GROUP BY gender;`

52

53 -- • Which day of the week has the highest average transaction amount in the cc_data table?

54

55

56

57

58

59

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

gender	avg_transaction_amount
F	69.9930655965829
M	72.07430999092087

Table: location_data

Columns:

cc_num	text
lat	text
long	text

Object Info Session Result 26 x Read Only

5.4 Which day of the week has the highest average transaction amount in the **cc_data** table?

Financial Fraud Detection_SQL*

Limit to 1000 rows

52

53 → • Which day of the week has the highest average transaction amount in the cc_data table?

54 • `SELECT DAYNAME(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS day_of_week, AVG(amt) AS avg_transaction_amount`

55 `FROM cc_data`

56 `GROUP BY day_of_week`

57 `ORDER BY avg_transaction_amount DESC`

58 `LIMIT 1;`

59

60

61

62

63

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |

day_of_week	avg_transaction_amount
Monday	72.4683134676563

Table: location_data

Columns:

cc_num	text
lat	text
long	text

Object Info Session Result 28 x Read Only